"SOLID

DRY
KISS
"
explain all them in details

SOLID, DRY, and KISS are foundational engineering principles that help teams write code that is easier to understand, change, test, and operate over time. Each principle targets a different source of complexity and risk, and together they promote clarity, modularity, and maintainability in software systems.[1] [2] [3]

SOLID

SOLID is a set of five object-oriented design principles that guide the structure of classes, interfaces, and dependencies so systems can evolve without constant breakage or rewrites. While rooted in OO, the ideas generalize to modular design in many languages and architectures, emphasizing separation of responsibilities, stable extension points, and dependency boundaries.[2] [3]

- Single Responsibility Principle (SRP): A class/module should have one reason to change, which reduces accidental coupling and makes refactoring, testing, and debugging more focused and predictable.[3] [2]
- Open/Closed Principle (OCP): Software entities should be open for extension but closed for modification, encouraging new behavior via composition or polymorphism rather than editing stable, tested code paths.[2] [3]
- Liskov Substitution Principle (LSP): Subtypes must be substitutable for their base types without altering correctness, preventing inheritance hierarchies where derived classes violate expected contracts.[3] [2]
- Interface Segregation Principle (ISP): Prefer many small, specific interfaces over large, fat ones so clients depend only on what they actually use, reducing needless coupling and changes rippling across consumers.[2] [3]
- Dependency Inversion Principle (DIP): Depend on abstractions, not concretions, so high-level policies don’t rely on low-level details, allowing implementations to change with minimal impact via inversion and injection.[3] [2]

DRY

DRY, “Don’t Repeat Yourself,” aims to eliminate duplication of knowledge and logic so there is a single source of truth, reducing bugs, change effort, and divergence over time. Duplication makes codebases larger and harder to reason about, since the same change must be applied consistently across multiple locations with error-prone coordination.[4] [1]

- What to deduplicate: Business rules, validation logic, data schemas, query construction, and configuration that encode the same concept more than once should be factored into shared functions/modules or authoritative data definitions.[1] [4]
- Practical application: Extract common behavior into well-named functions or services, prefer shared libraries over copy-paste, centralize constants and schemas, and ensure tests pin the single canonical behavior rather than multiple near-copies.[4] [1]
- Balance: DRY must be applied thoughtfully—premature abstraction of similar-but-not-identical code can create brittle “false DRY,” so deduplicate only when intent and variability are well understood.[1] [4]

KISS

KISS, “Keep It Simple, Stupid” (also “Keep It Short and Simple”), prioritizes the simplest solution that meets requirements, avoiding unnecessary abstraction, premature optimization, and over-engineering that increase risk and maintenance cost. Simple designs are easier to read, test, modify, and scale because they reduce hidden coupling and cognitive load for developers and operators.[5] [6] [1]

- Practical implications: Prefer straightforward designs, clear naming, and minimal moving parts; delay complex frameworks or extensibility until a concrete need emerges; and iterate by refactoring toward simpler forms after a working baseline exists.[6] [5]
- Benefits: Simplicity improves clarity and usability, reduces defects and performance pitfalls, and makes future changes safer and faster by limiting the blast radius of modifications.[5] [1]
- Tips: Avoid excessive abstractions, keep functions/classes small and focused, choose readable solutions over clever tricks, and resist premature optimization unless profiling shows a clear hotspot.[6] [1]

How they work together

Used together, SOLID shapes modular structure, DRY reduces duplicated knowledge within that structure, and KISS constrains complexity so abstractions remain proportionate to real needs, producing systems that are simple to understand yet robust to change. A healthy balance avoids the extremes of over-abstraction (violating KISS) and copy-paste proliferation (violating DRY), while SOLID provides guide rails for organizing code so extension and substitution don’t destabilize the system.[2] [3]

⁂

Compare SOLID, DRY, and KISS with examples in code

SOLID, DRY, and KISS address different dimensions of code quality: SOLID guides how to structure modules and dependencies, DRY eliminates duplicated knowledge, and KISS keeps solutions as simple as the problem demands.[21] [22] [23]

Quick comparison

- SOLID: Organize classes and dependencies to be extendable without breaking existing code, especially in OO designs.[24] [21]
- DRY: Ensure every piece of knowledge has a single, unambiguous representation to avoid divergence and extra maintenance.[23] [25]
- KISS: Prefer the simplest design that meets requirements; avoid over-engineering and premature abstraction.[26] [27]

SOLID with code

Goal: Enable safe extension and evolution by clear responsibilities, substitutable abstractions, focused interfaces, and inverted dependencies.[21] [24]

- SRP (Single Responsibility)
  Bad: One class both performs business logic and logs errors.

```
class Customer {
    void Add(Database db) {
        try { db.Add(); }
        catch (Exception ex) {
            File.WriteAllText(@"C:\Error.txt", ex.ToString()); // logging mixed in
        }
    }
}
```

Better: Separate logging responsibility.

```
class Customer {
    private readonly ILogger logger;
    public Customer(ILogger logger) { this.logger = logger; }

    void Add(Database db) {
        try { db.Add(); }
        catch (Exception ex) { logger.Handle(ex.ToString()); }
    }
}

interface ILogger { void Handle(string error); }
class FileLogger : ILogger {
    public void Handle(string error) => File.WriteAllText(@"C:\Error.txt", error);
}
```

This isolates logging, making Customer change only for customer concerns, not log I/O details. [28] [24]

- OCP (Open/Closed)
  Add new discount strategies without modifying checkout logic.

```
interface Discount { double apply(double amount); }

class NoDiscount implements Discount { public double apply(double a){ return a; } }
class PercentageDiscount implements Discount {
    private final double rate;
    public PercentageDiscount(double rate){ this.rate = rate; }
    public double apply(double a){ return a * (1 - rate); }
}

class Checkout {
    private final Discount discount;
    public Checkout(Discount discount){ this.discount = discount; }
    public double total(double amount){ return discount.apply(amount); }
}
```

New discounts extend via new classes; checkout code remains closed to modification. [24] [21]

- LSP (Liskov Substitution)
  Subtypes must honor base contracts. A Rectangle/Square trap illustrates violations.

```
class Rectangle:
    def __init__(self, w, h): self.w, self.h = w, h
    def set_width(self, w): self.w = w
    def set_height(self, h): self.h = h
    def area(self): return self.w * self.h

class Square(Rectangle):
    def set_width(self, w):
        self.w = self.h = w
    def set_height(self, h):
        self.w = self.h = h
```

Code expecting a Rectangle that can change width independently breaks with Square; substitutability is violated, so prefer composition or separate interfaces to maintain contracts. [21] [24]

- ISP (Interface Segregation)
  Small, focused interfaces for clients' needs.

```
interface Printer { print(doc: string): void; }
interface Scanner { scan(): string; }

// Clients depend only on what they use.
class SimplePrinter implements Printer { print(doc: string){ /*...*/ } }
class MultiFunction implements Printer, Scanner { /*...*/ }
```

Avoid “fat” interfaces that force unused methods on clients.[24] [21]

- DIP (Dependency Inversion)
  High-level policy depends on abstractions, not concretes.

```
interface PaymentGateway { fun charge(cents: Int): Boolean }

class StripeGateway: PaymentGateway { override fun charge(c: Int)=true }
class OrderService(private val gateway: PaymentGateway) {
    fun place(amount: Int): Boolean = gateway.charge(amount)
}
```

Swapping gateways requires no changes to OrderService; injection chooses the implementation.[21] [24]

DRY with code

Goal: One canonical place for each rule to avoid inconsistency and duplicated changes.[25] [23]

- Before (duplicated logic):

```
function priceAfterDiscountA(price) {
  const discountA = 0.1;
  return price * (1 - discountA);
}
function priceAfterDiscountB(price) {
  const discountB = 0.2;
  return price * (1 - discountB);
}
```

After (single, parameterized rule):

```
function priceAfterDiscount(price, rate) {
  return price * (1 - rate);
}
```

A single function now encodes the discount rule; callers pass the rate they need.[29] [30]

- Another refactor example (unify similar transforms):
  Before:

```
def calculate_price_with_tax(price, tax_rate):
    return price + price * tax_rate

def calculate_discounted_price(price, discount_rate):
    return price - price * discount_rate
```

After:

```
def calculate_price(price, rate, is_tax=True):
    return price + price * rate if is_tax else price - price * rate
```

One function captures the shared computation, reducing duplication and divergence risk.[30] [25]

- Caution: Avoid “false DRY” by merging code that looks similar but differs in semantics; abstract only when variability and intent are clear.[23] [25]

KISS with code

Goal: Prefer straightforward solutions; avoid over-engineering and unnecessary abstractions.[27] [26]

- Over-engineered vs simple:
  Non-KISS (too many layers for a counter):

```
// Overkill abstractions for a simple counter
type IncrementStrategy interface { Next(int) int }
type ByOne struct{}
func (ByOne) Next(n int) int { return n + 1 }

type Counter struct { n int; s IncrementStrategy }
func NewCounter(s IncrementStrategy) *Counter { return &Counter{0, s} }
func (c *Counter) Inc() { c.n = c.s.Next(c.n) }
```

KISS (direct, readable):

```
type Counter struct { n int }
func (c *Counter) Inc() { c.n++ }
```

The simple version meets the need with fewer moving parts, easier to test and maintain.[26] [27]

- KISS in conditionals:
  Non-KISS:

```
if (user.is_admin and not user.is_guest) or (user.role in ["OWNER","SUPER"] and feature.e
    grant_access()
```

KISS:

```
def can_access(user, feature):
    if user.is_banned: return False
    if user.is_admin: return True
    return feature.enabled and user.role in {"OWNER","SUPER"}

if can_access(user, feature):
    grant_access()
```

Extracting intent into a small function and readable steps lowers cognitive load without changing behavior.[27] [26]

How to choose which principle

- If the problem is duplicated rules or copy-paste: reach for DRY first and centralize the knowledge behind a clear API or function.[25] [23]
- If extension keeps breaking old code or inheritance/substitution quirks appear: apply SOLID to define stable abstractions, segregated interfaces, and inverted dependencies.[24] [21]
- If the design feels heavy for the use case or is hard to read: apply KISS; remove unnecessary layers, keep functions/classes small, and delay abstractions until a real need exists.[26] [27]

Putting them together

A balanced approach: start with KISS to ship a clear baseline, refactor toward DRY to remove duplication as patterns emerge, and apply SOLID to shape extension points and dependency boundaries when requirements grow—avoiding both over-abstraction and copy-paste creep.[23] [21]

⁂

in MERN base + mysql

Here's how to apply and compare SOLID, DRY, and KISS in a MERN-style app where the backend is Node/Express with MySQL (using mysql2 or Sequelize) and the frontend is React. Code examples show the same feature (Todos) implemented with each principle in mind.[41] [42] [43]

What changes in MERN + MySQL

- Replace MongoDB with MySQL in the Node/Express layer using a driver (mysql2/promise) or an ORM (Sequelize), while React and Express patterns remain similar.[43] [41]
- Typical stack: React (axios) → Express routes/controllers → service/use-case → repository (SQL) with mysql2 or Sequelize models.[42] [43]

KISS first: keep it simple

Start with the simplest working version, then refactor as needs grow.[41] [43]

- Minimal API for a Todo list (no abstractions yet):

```
// server.js
import express from 'express';
import mysql from 'mysql2/promise';

const app = express();
app.use(express.json());
```

```
const pool = await mysql.createPool({
  host: 'localhost', user: 'root', password: 'pass', database: 'todos_db'
});

app.get('/api/todos', async (req, res) => {
  const [rows] = await pool.query('SELECT id, title, done FROM todos ORDER BY id DESC');
  res.json(rows);
});

app.post('/api/todos', async (req, res) => {
  const { title } = req.body;
  const [result] = await pool.query('INSERT INTO todos (title, done) VALUES (?, ?)', [tit
  res.status(201).json({ id: result.insertId, title, done: 0 });
});

app.listen(3000);
```

This is clear and shippable; avoid extra layers until requirements prove they're needed.[43] [41]

- React KISS fetch:

```
// Todos.jsx
import { useEffect, useState } from 'react';
import axios from 'axios';

export default function Todos() {
  const [todos, setTodos] = useState([]);
  const [title, setTitle] = useState('');

  useEffect(() => { axios.get('/api/todos').then(r => setTodos(r.data)); }, []);
  const add = async () => {
    const { data } = await axios.post('/api/todos', { title });
    setTodos([data, ...todos]); setTitle('');
  };

  return (
    <>
      <input value={title} onChange={e => setTitle(e.target.value)} />
      <button onClick={add}>Add</button>
      <ul>{todos.map(t => <li key={t.id}>{t.title}</li>)}</ul>
    </>
  );
}
```

Small, readable, minimal moving parts: this satisfies KISS for an MVP.[44] [41]

DRY: remove duplication as patterns emerge

When logic repeats across routes (validation, mapping, SQL), centralize it so there's one source of truth.[45] [41]

- Before (duplication): repeated manual mapping and queries in each handler.

- After (DRY service + repository):

```
// db/pool.js
import mysql from 'mysql2/promise';
export const pool = await mysql.createPool({ host: 'localhost', user: 'root', password: '

// repos/todoRepo.js
import { pool } from '../db/pool.js';
export async function listTodos() {
  const [rows] = await pool.query('SELECT id, title, done FROM todos ORDER BY id DESC');
  return rows;
}
export async function createTodo(title) {
  const [r] = await pool.query('INSERT INTO todos (title, done) VALUES (?, ?)', [title, 0
  return { id: r.insertId, title, done: 0 };
}

// services/todoService.js
export function validateTitle(title) {
  if (!title || title.length < 2) throw new Error('Title too short');
  return title.trim();
}
export async function addTodo(title) { return createTodo(validateTitle(title)); }

// routes/todoRoutes.js
import express from 'express';
import { listTodos } from '../repos/todoRepo.js';
import { addTodo } from '../services/todoService.js';
const router = express.Router();

router.get('/', async (req, res, next) => {
  try { res.json(await listTodos()); } catch (e) { next(e); }
});
router.post('/', async (req, res, next) => {
  try { res.status(201).json(await addTodo(req.body.title)); } catch (e) { next(e); }
});
export default router;

// server.js
import express from 'express';
import todoRoutes from './routes/todoRoutes.js';
const app = express();
app.use(express.json());
app.use('/api/todos', todoRoutes);
app.listen(3000);
```

- Validation, queries, and mapping live in one place, so changes happen once; handlers are thin and consistent, reducing copy-paste and drift.[42] [43]
- DRY in React: centralize API calls and types.

```
// api/todos.js
import axios from 'axios';
```

```
export const getTodos = () => axios.get('/api/todos').then(r => r.data);
export const addTodo = (title) => axios.post('/api/todos', { title }).then(r => r.data);
```

Components use the same API wrappers; schema changes update a single module.[44] [41]

SOLID: structure for safe extension

Introduce abstractions when evolving requirements: new storage, new rules, background jobs, etc. Use SOLID to keep modules cohesive, extendable, and loosely coupled.[46] [47]

- SRP: Controllers only handle HTTP, services encapsulate business rules, repositories handle data access; each layer has one reason to change.[47] [46]
- OCP + DIP: Depend on interfaces so you can switch from MySQL to another repo (or to Sequelize) without touching services/controllers.[46] [47]

```
// ports/TodoRepository.ts (TypeScript for clarity)
export interface TodoRepository {
  list(): Promise<Todo[]>;
  create(title: string): Promise<Todo>;
}

// adapters/mysql/MySqlTodoRepository.ts
import { pool } from '../../db/pool';
export class MySqlTodoRepository implements TodoRepository {
  async list() { const [rows] = await pool.query('SELECT id,title,done FROM todos'); retu
  async create(title: string) {
    const [r] = await pool.query('INSERT INTO todos (title,done) VALUES (?,?)', [title, 0
    return { id: r.insertId, title, done: 0 };
  }
}

// services/TodoService.ts
export class TodoService {
  constructor(private repo: TodoRepository) {}
  add(title: string) {
    if (!title || title.length < 2) throw new Error('bad title');
    return this.repo.create(title.trim());
  }
  list() { return this.repo.list(); }
}

// composition root (server.ts)
const repo = new MySqlTodoRepository();
const service = new TodoService(repo);
```

Service depends on the abstraction; adding a Postgres or in-memory repo means new class, no edits to business logic (Open for extension, closed for modification; depends on abstraction per DIP).[47] [46]

- LSP: Alternate repos must fulfill the same contract (list/create semantics, error handling) so controllers/services aren’t surprised by incompatible behavior.[46] [47]

- ISP: Split interfaces so consumers depend only on what they use (e.g., QueryRepository with list, CommandRepository with create/update), especially helpful when scaling read/write paths differently.[47] [46]

```
export interface TodoReader { list(): Promise<Todo[]> }
export interface TodoWriter { create(title: string): Promise<Todo> }
```

- Using Sequelize with SOLID:

```
// models/todo.model.js (Sequelize)
export default (sequelize, DataTypes) => {
  return sequelize.define('todo', {
    title: { type: DataTypes.STRING, allowNull: false },
    done: { type: DataTypes.BOOLEAN, defaultValue: false }
  });
};
```

This keeps SQL concerns encapsulated in the adapter; services stay ORM-agnostic and testable with in-memory fakes.[42] [43]

Practical tips in this stack

- KISS: Start with mysql2/promise and a single router; avoid premature generic repositories or an over-abstracted service layer until requirements justify it.[41] [43]
- DRY: Centralize validation, error mapping, and DTOs; create one error handler middleware that formats DB errors consistently across routes.[43] [42]
- SOLID: Introduce ports/adapters when adding a second data source, background worker, or external service; use dependency injection via a simple composition module or a light DI container if needed later.[46] [47]

Example DB connection patterns

- mysql2/promise pool (simple and KISS):

```
import mysql from 'mysql2/promise';
export const pool = await mysql.createPool({ host:'localhost', user:'root', password:'pas
```

Reliable, fast to set up, widely used in tutorials and guides for Express + MySQL.[48] [43]

- Sequelize (adds migrations/relations, good for SOLID layering):
  Install: npm i sequelize mysql2; define models and use repository adapters around Sequelize calls for DIP/OCP.[42] [43]

When to apply which

- Start KISS to deliver value quickly and avoid unnecessary layers. As repeated logic appears, refactor to DRY. When change pressure increases (multiple data sources, complex rules), adopt SOLID to protect stability and enable extension without churn.[45] [46]
- For a student or small team, a two-layer split (routes/controllers + repository) is often enough; add services and interfaces as domain complexity grows to keep code evolvable without regressions.[41] [43]

⁂

how to create a register and log in page with react and tailwind

Below is a step-by-step approach to create responsive Register and Login pages in React with Tailwind, including form structure, validation options, and ready-to-adapt code snippets, plus references to high-quality UI blocks and templates you can copy-paste and customize quickly.[62] [63]

Project setup

- Initialize a React app and add Tailwind CSS following Tailwind's standard setup, then confirm Tailwind classes render correctly by trying a minimal styled div before building forms.[64] [65]
- For fastest UI scaffolding, start from Tailwind's prebuilt sign-in/register blocks or component libraries like Flowbite/TW Elements, which provide accessible, responsive forms out of the box to customize with your brand.[65] [62]

Login form (basic Tailwind)

- This minimal example uses semantic inputs and Tailwind utilities; you can enhance validation later with react-hook-form or custom logic as needs grow for KISS compliance.[63] [65]

```
// src/pages/Login.jsx
import { useState } from "react";

export default function Login() {
  const [form, setForm] = useState({ email: "", password: "" });
  const onChange = (e) => setForm({ ...form, [e.target.name]: e.target.value });
  const onSubmit = async (e) => {
    e.preventDefault();
    // TODO: call your API (e.g., /api/auth/login) and handle tokens
  };

  return (
    <main className="min-h-screen flex items-center justify-center bg-gray-50">
      <form onSubmit={onSubmit} className="w-full max-w-sm bg-white p-6 rounded-lg shadow
        <h1 className="text-xl font-semibold mb-4">Sign in</h1>

        <label className="block mb-3">
```

```
          <span className="text-sm text-gray-700">Email</span>
          <input
            type="email"
            name="email"
            value={form.email}
            onChange={onChange}
            required
            className="mt-1 w-full rounded-md border border-gray-300 px-3 py-2 focus:outl
            placeholder="you@example.com"
          />
        </label>

        <label className="block mb-4">
          <span className="text-sm text-gray-700">Password</span>
          <input
            type="password"
            name="password"
            value={form.password}
            onChange={onChange}
            required
            className="mt-1 w-full rounded-md border border-gray-300 px-3 py-2 focus:outl
            placeholder="••••••••"
          />
        </label>

        <button
          type="submit"
          className="w-full rounded-md bg-indigo-600 text-white py-2 hover:bg-indigo-700
        >
          Sign in
        </button>
      </form>
    </main>
  );
}
```

- For design shortcuts, Tailwind UI's Sign-in and Registration gallery provides polished layouts; copy a block and adapt labels, validation hints, and brand colors as needed.[62] [65]

Registration form (basic Tailwind)

- Start with name, email, password, and confirm password, using required attributes and clear error text areas that you can fill once validation is wired.[66] [65]

```
// src/pages/Register.jsx
import { useState } from "react";

export default function Register() {
  const [form, setForm] = useState({ name: "", email: "", password: "", confirm: "" });
  const [errors, setErrors] = useState({});
  const onChange = (e) => setForm({ ...form, [e.target.name]: e.target.value });

  const validate = () => {
    const e = {};
    if (!/^\S+@\S+\.\S+$/.test(form.email)) e.email = "Invalid email";
```

```
    if (form.password.length < 8) e.password = "Min 8 characters";
    if (form.confirm !== form.password) e.confirm = "Passwords do not match";
    setErrors(e);
    return Object.keys(e).length === 0;
  };

  const onSubmit = async (ev) => {
    ev.preventDefault();
    if (!validate()) return;
    // TODO: call your API (e.g., /api/auth/register)
  };

  return (
    <main className="min-h-screen flex items-center justify-center bg-gray-50">
      <form onSubmit={onSubmit} className="w-full max-w-sm bg-white p-6 rounded-lg shadow
        <h1 className="text-xl font-semibold mb-4">Create account</h1>

        <label className="block mb-3">
          <span className="text-sm text-gray-700">Name</span>
          <input
            name="name"
            value={form.name}
            onChange={onChange}
            required
            className="mt-1 w-full rounded-md border border-gray-300 px-3 py-2 focus:outl
            placeholder="Your name"
          />
        </label>

        <label className="block mb-3">
          <span className="text-sm text-gray-700">Email</span>
          <input
            type="email"
            name="email"
            value={form.email}
            onChange={onChange}
            required
            className="mt-1 w-full rounded-md border border-gray-300 px-3 py-2 focus:outl
            placeholder="you@example.com"
          />
          {errors.email && <p className="text-red-600 text-sm mt-1">{errors.email}</p>}
        </label>

        <label className="block mb-3">
          <span className="text-sm text-gray-700">Password</span>
          <input
            type="password"
            name="password"
            value={form.password}
            onChange={onChange}
            required
            className="mt-1 w-full rounded-md border border-gray-300 px-3 py-2 focus:outl
            placeholder="At least 8 characters"
          />
          {errors.password && <p className="text-red-600 text-sm mt-1">{errors.password}<
        </label>
```

```
        <label className="block mb-4">
          <span className="text-sm text-gray-700">Confirm password</span>
          <input
            type="password"
            name="confirm"
            value={form.confirm}
            onChange={onChange}
            required
            className="mt-1 w-full rounded-md border border-gray-300 px-3 py-2 focus:outl
            placeholder="Repeat password"
          />
          {errors.confirm && <p className="text-red-600 text-sm mt-1">{errors.confirm}</p
        </label>

        <button
          type="submit"
          className="w-full rounded-md bg-indigo-600 text-white py-2 hover:bg-indigo-700
        >
          Create account
        </button>
      </form>
    </main>
  );
}
```

- For more complex UIs, you can start from templates such as Flowbite/TW Elements or curated Tailwind login/register collections to speed up development and ensure responsive behavior across devices.[67] [68]

Adding react-hook-form and schema validation

- For robust UX and fewer re-renders, integrate react-hook-form with a schema validator like Yup/Zod to declaratively enforce rules and display errors with Tailwind classes.[69] [63]

```
// src/pages/LoginHookForm.jsx
import { useForm } from "react-hook-form";

export default function LoginHookForm() {
  const { register, handleSubmit, formState: { errors, isSubmitting } } = useForm();
  const onSubmit = async (data) => {
    // { email, password } -> API call
  };

  return (
    <main className="min-h-screen flex items-center justify-center bg-gray-50">
      <form onSubmit={handleSubmit(onSubmit)} className="w-full max-w-sm bg-white p-6 rou
        <h1 className="text-xl font-semibold mb-4">Sign in</h1>

        <label className="block mb-3">
          <span className="text-sm text-gray-700">Email</span>
          <input
            className={`mt-1 w-full rounded-md border px-3 py-2 focus:outline-none focus:
            {...register("email", { required: "Email is required", pattern: { value: /^\S
```

```
            placeholder="you@example.com"
            type="email"
          />
          {errors.email && <p className="text-red-600 text-sm mt-1">{errors.email.message
        </label>

        <label className="block mb-4">
          <span className="text-sm text-gray-700">Password</span>
          <input
            className={`mt-1 w-full rounded-md border px-3 py-2 focus:outline-none focus:
            {...register("password", { required: "Password is required", minLength: { val
            placeholder="••••••••"
            type="password"
          />
          {errors.password && <p className="text-red-600 text-sm mt-1">{errors.password.m
        </label>

        <button disabled={isSubmitting} type="submit" className="w-full rounded-md bg-inc
          {isSubmitting ? "Signing in..." : "Sign in"}
        </button>
      </form>
    </main>
  );
}
```

- You can also style validation purely with Tailwind's invalid/peer classes, but react-hook-form gives better programmatic control and error messaging patterns in larger apps.[70] [69]

Production tips

- Accessibility: Pair inputs with labels, use focus-visible rings, and provide error text with clear contrast for keyboard and screen reader users to meet good UX standards.[65] [62]
- Reusability: Extract Input and Button components to keep pages DRY; Tailwind component libraries and UI blocks can serve as a starting point for your design language across auth flows.[62] [65]
- Templates: If you want to move fast, copy one of the ready React + Tailwind login/register templates and iterate with your branding, validation, and API calls.[71] [67]

Ready-made UI resources

- Tailwind UI Blocks for sign-in/registration provide polished, responsive sections you can paste into React and wire to state and handlers in minutes.[65] [62]
- Flowbite/TW Elements offer React-ready login/register examples and validation patterns you can adapt without building from scratch.[72] [73]

Next step: hook to your API

- On submit, call your backend endpoints (e.g., POST /api/auth/register and POST /api/auth/login), store returned tokens securely (httpOnly cookies are recommended), and show inline error messages based on API responses while keeping the UI responsive with Tailwind states.[63] [65]

⁂

"Git commands. Git merge, git rebase, checkout, branch management, pull requests, issue creation."

explain them also give video and image

Git merge, rebase, checkout/switch, branch management, pull requests, and issues are core workflows for collaboration and clean histories. Below are concise explanations, recommended commands, and handpicked video resources, plus an illustrative image note at the end.[82] [83]

Git merge vs rebase

- Merge combines another branch into the current one by creating a merge commit, preserving the exact history as it happened, which is ideal on shared/public branches.[83] [82]
- Rebase rewrites the feature branch to start from a newer base, producing a linear history; it's best on private branches you alone control to avoid rewriting shared history.[82] [83]

Common commands:

- Merge:
  - git checkout main; git merge feature/login to bring feature into main with a merge commit.[82]
- Rebase:
  - git checkout feature/login; git rebase main to replay your commits on top of main for a linear history.[82]

When to choose:

- Prefer merge for shared branches and to keep the audit trail intact; choose rebase to clean up personal feature branches before merging.[84] [83]

git checkout, switch, restore

- git checkout historically handled both branch switching and file checkout, which caused ambiguity; modern Git recommends git switch for branches and git restore for files, though checkout still works.[85] [86]
- Examples:

- Switch to existing branch: git switch main (or git checkout main).[87] [85]
- Create and switch: git switch -c feature/auth (or git checkout -b feature/auth).[85] [87]
- Restore a file from HEAD: git restore path/to/file (previously git checkout -- path).[85]

Tips:

- If switching branches fails due to local changes, commit/stash first, or use git stash push -m "wip" and switch safely, as described in practical guides.[87]

Branch management

- Create, list, delete:
  - List: git branch (local), git branch -r (remote), git branch -a (all).[88] [87]
  - Create: git switch -c feature/payments; push upstream: git push -u origin feature/payments.[87] [85]
  - Delete: git branch -d feature/payments (or -D to force after merged status checks).[88]
- Keep feature branches up to date:
  - Merge main into your branch: git switch feature; git merge main, or rebase: git switch feature; git rebase main based on your team policy.[83] [82]

Pull requests (PRs)

- A PR proposes changes from a branch to the target branch for review; it bundles commits, diffs, and discussion before merge on platforms like GitHub.[89]
- Good PR hygiene: clear title/description, linked issue, small focused diffs, and labels/milestones; these improve async coordination and automation in CI/CD workflows.[89]
- Typical flow:
  1. git switch -c feature/xyz; commit work; git push -u origin feature/xyz.
  2. Open PR on GitHub, request reviews, address feedback, and merge via merge, squash, or rebase per repo rules.[89]

Issues (GitHub)

- Issues capture bugs, tasks, and proposals with reproducible steps, expected behavior, environment, and acceptance criteria; they power tracking, labels, assignees, and automation.[89]
- Use saved views and filters to track assigned/created/mentioned issues and PRs across repositories efficiently in the GitHub dashboard.[90]

Recommended videos

- Git MERGE vs REBASE: Everything You Need to Know — practical, visual explanation of history outcomes and when to use each.[91]
- Intro to GitHub: Commits, Issues, Pull Requests, Releases — a beginner-friendly overview of GitHub collaboration primitives.[92]
- GitHub Workflow for Issues and Pull Requests — step-by-step demonstration of opening issues, creating branches, and PR review/merge.[93]

Handy image note

- The Atlassian tutorial's merge vs rebase article includes diagrams contrasting non-linear merge histories with linearized rebase histories, which is useful as a quick visual reference when choosing a strategy for your team.[82]

Quick command cheatsheet

- New branch and push: git switch -c feature/auth && git push -u origin feature/auth.[85]
- Merge feature into main: git switch main && git merge feature/auth && git push.[82]
- Rebase feature onto main: git switch feature/auth && git fetch origin && git rebase origin/main.[83] [82]
- Resolve rebase conflicts stepwise: fix files → git add -A → git rebase --continue; abort with git rebase --abort if needed.[82]
- Open PR: push branch, then use GitHub UI to compare feature/auth → main and open a pull request with description and linked issue.[89]
- Create issue: in GitHub, New issue → fill title/body with steps, expected vs actual, environment, and add labels/assignee for triage and tracking.[90] [89]

⁂

"SOLID

DRY
KISS
API
Git and github
*Modular architecure
*REST API architecture

"
make a short note above the topic also explain in real life term

Here are short, practical notes on each topic with simple real-life analogies to cement understanding. Each item ends with a one-line takeaway you can use for revision.

SOLID

- Short note: A set of five OO design principles that keep code modular, testable, and easy to extend without breaking existing behavior.
- Real life: Think of a restaurant kitchen where each station has a single job (SRP), new dishes are added via specials without rewriting the main menu (OCP), any trained chef can cover a station without surprises (LSP), stations have precise toolsets instead of one giant toolbox (ISP), and the head chef sets recipes while line cooks follow abstracted steps rather than hard-coding personal tricks (DIP).
- Takeaway: Structure code so responsibilities are focused, extensions are easy, and dependencies are abstracted.

DRY

- Short note: Don’t Repeat Yourself—put each piece of knowledge in one canonical place to avoid divergence and extra maintenance.
- Real life: One central recipe card in a kitchen instead of three versions scribbled on sticky notes; change the card once and every cook gets the correct steps.
- Takeaway: Extract common logic into shared functions/modules and update once, everywhere.

KISS

- Short note: Keep It Simple—choose the simplest design that fully solves the problem; avoid premature abstractions or cleverness.
- Real life: For a short trip, pack a backpack, not a suitcase set with organizers; complexity adds weight and slows you down.
- Takeaway: Prefer straightforward solutions; only add layers when real needs appear.

API

- Short note: An Application Programming Interface defines how software components communicate via stable contracts—methods, data formats, and status codes.
- Real life: A restaurant menu as a contract—you order by item and options, the kitchen returns the dish; you don’t enter the kitchen or dictate internal steps.
- Takeaway: Design clear, versioned contracts that hide internals and provide predictable responses.

Git and GitHub

- Short note: Git tracks code history through commits and branches; GitHub hosts repos for collaboration with pull requests, reviews, and issues.
- Real life: Git is your personal time machine and branching notebook; GitHub is the shared workspace where teammates propose changes (PRs) and file tasks/bugs (issues).

- Takeaway: Commit small, branch for features, open PRs for review, and track work with issues.

Modular architecture

- Short note: Build systems from well-defined modules with explicit interfaces so parts can be developed, tested, and replaced independently.
- Real life: A PC built from swappable parts—CPU, RAM, GPU—each with standardized connectors; upgrade one without rebuilding the whole machine.
- Takeaway: Encapsulate features per module, expose only what's needed, and keep dependencies directional.

REST API architecture

- Short note: A resource-oriented style over HTTP using nouns (resources), standard methods (GET/POST/PUT/PATCH/DELETE), and status codes; stateless requests with consistent representations (often JSON).
- Real life: Library system—books (resources) are fetched, created, updated, or removed via clear actions at known counters; each request carries all info needed, the librarian doesn't remember your last visit.
- Takeaway: Model resources cleanly, use correct verbs and status codes, keep requests stateless and predictable.

Quick command/action crib

- Git basics:
  - Create branch: git switch -c feature/auth
  - Commit: git add -A && git commit -m "feat: add auth"
  - Merge main into feature: git switch feature/auth && git merge main
  - Rebase feature on main: git switch feature/auth && git rebase main
  - Open PR: push branch and create PR on the platform
- REST basics:
  - GET /users → list users
  - POST /users → create user
  - GET /users/{id} → fetch user
  - PATCH /users/{id} → partial update
  - DELETE /users/{id} → remove user

How they fit together

- Start with KISS to deliver a simple baseline.
- Refactor to DRY as repetition appears.
- Shape modules and extension points with SOLID and modular architecture.
- Expose capabilities through a clean REST API and manage all changes with Git/GitHub workflows.

If you want, a one-page printable version can be generated with condensed bullets and a small diagram mapping “KISS → DRY → SOLID” across project phases, plus a REST verbs table and Git flow arrows.

⁂

explain all more details and make a pdf

Here’s a detailed, practical guide to SOLID, DRY, KISS, API, Git/GitHub, Modular Architecture, and REST API Architecture, with real-life analogies and quick checklists. A downloadable PDF can be provided—reply if a styled PDF export is preferred in A4 with headings, code blocks, and a one-page cheatsheet.

SOLID

- What it is: Five OO design principles shaping maintainable, extensible code: SRP, OCP, LSP, ISP, DIP. These reduce coupling, improve testability, and allow safe change over time.[112] [113]
- Real life: A restaurant where each station handles a single task (SRP), new dishes are added as specials without rewriting the menu (OCP), any trained chef can substitute at a station without breaking workflow (LSP), stations have focused tools not giant all-in-one contraptions (ISP), and recipes are abstracted so head chef defines policy while cooks follow it (DIP).[114] [115]
- Checklist:
  - One reason to change per module (SRP).
  - Extend via new classes/plugins, not edits to stable core (OCP).
  - Subtypes keep contracts and invariants (LSP).
  - Small, focused interfaces (ISP).
  - High-level depends on abstractions; inject implementations (DIP).[112] [114]

DRY

- What it is: Don’t Repeat Yourself—one canonical source for each rule, schema, or operation. Duplication multiplies bugs and change cost.[113] [114]
- Real life: One master recipe card used by all cooks; update once, everyone follows the same steps.[114]

- Checklist:
  - Factor shared logic; avoid copy-paste.
  - Centralize constants, schemas, validation rules.
  - Watch out for “false DRY” (abstracting things that only look similar).[116] [114]

KISS

- What it is: Keep It Simple—choose the simplest design that meets requirements; avoid over-engineering and cleverness that raise maintenance cost.[115] [114]
- Real life: For a short trip, take a backpack, not three suitcases with organizers.[114]
- Checklist:
  - Prefer straightforward code and readable names.
  - Start simple; add abstractions only when needed.
  - Optimize only after profiling shows hotspots.[114]

API

- What it is: A stable contract for software interaction—endpoints/methods, request/response formats, and error codes. Hides internals behind predictable interfaces.[117] [118]
- Real life: A restaurant menu: you order by name and options; the kitchen returns the dish without exposing internal steps.[118]
- Checklist:
  - Clear contract and versioning.
  - Consistent data shapes and error handling.
  - Backwards compatibility during evolution.[119] [117]

Git and GitHub

- What it is: Git is distributed version control (commits, branches, merges). GitHub adds hosting, pull requests, code review, issues, and workflows.[120] [121]
- Real life: Git is a time machine for your work and branching notepads for experiments; GitHub is the shared office where teammates review changes and track tasks.[121]
- Core workflows:
  - Branch strategies: GitHub Flow (simple feature branches + PRs), Trunk-based (short-lived branches), GitFlow (feature, develop, release, hotfix).[120] [121]
  - PR hygiene: small focused diffs, clear descriptions, linked issues, CI checks; squash merges to keep history tidy if team policy allows.[122] [123]
  - Basics:
    - Create branch: git switch -c feature/auth; push: git push -u origin feature/auth.

    - Merge main into feature: git merge main; or rebase: git rebase main (use rebase only on private branches).[123] [120]

Modular architecture

- What it is: Split systems into cohesive modules with explicit interfaces and minimal coupling so parts can be built, tested, and replaced independently.[112]
- Real life: A desktop PC with standardized connectors; swap the GPU without rebuilding the whole machine.[112]
- Checklist:
  - Clear boundaries and contracts between modules.
  - Directional dependencies; avoid circular references.
  - Encapsulate internals; expose only what clients need.[112]

REST API architecture

- What it is: Resource-oriented design over HTTP using standard verbs (GET, POST, PUT, PATCH, DELETE), status codes, stateless requests, and consistent representations (often JSON).[124] [119]
- Real life: A library counter with clear actions: view, borrow, return; each request carries all info needed; the librarian doesn't remember your last visit (stateless).[124]
- Core elements:
  - Nouns for resources: /users, /users/{id}, /orders/{id}/items.[124]
  - Verbs: GET (read), POST (create), PUT (replace), PATCH (partial update), DELETE (remove).[125] [124]
  - Status codes: 200 OK, 201 Created, 204 No Content, 400 Bad Request, 401 Unauthorized, 403 Forbidden, 404 Not Found, 409 Conflict, 422 Unprocessable Entity, 500 Server Error.[117] [119]
  - Versioning: /v1/...; evolve without breaking clients.[119]
  - Validation and errors: Consistent error envelope with code, message, details.[117] [119]

How these fit together

- Start with KISS to ship a minimal, working API and simple GitHub Flow.
- Refactor to DRY as duplication appears in models, validation, and handlers.
- Shape modules and extension points using SOLID within a modular architecture to scale features safely.
- Expose capabilities via a clean, versioned REST API; use Git/GitHub workflows to collaborate, review, and release.[119] [112]

Quick practical patterns

- Git/GitHub Flow:
  - Branch: git switch -c feature/x; commit small; push and open a PR; ensure CI passes; squash merge; delete branch.[121] [123]
- REST endpoint design:
  - GET /users → list; POST /users → create; GET /users/{id} → fetch; PATCH /users/{id} → partial update; DELETE /users/{id} → delete; use 201 with Location header on create.[117] [119]
- Consistent error shape:
  - { "error": { "code": "VALIDATION_ERROR", "message": "Email invalid", "field": "email" } } with 422 status.[119] [117]

PDF delivery

- A PDF can be generated with:
  - One-page cheatsheet (verbs/status codes table, Git flows diagram, SOLID bullets).
  - Full notes with analogies, checklists, and examples in A4, dark-mode friendly.
- Say “Generate the PDF” and specify: A4 or Letter, font size, and whether to include diagrams. The PDF will be prepared accordingly.

⁂

generate the pdf

A polished PDF can be generated, but file export isn’t available directly in this chat. Here are two quick, reliable ways to get your PDF right now, plus a ready-to-copy Markdown source of the full notes you requested.

Option 1: VS Code method

- Install extensions: “Markdown All in One” and “Markdown PDF”.
- Create a file notes.md, paste the content below, then use Command Palette → “Markdown PDF: Export (pdf)”.
- This preserves headings, code, lists, and page breaks well.

Option 2: Browser converter method

- Paste the content below into any Markdown-to-PDF web converter.
- Choose A4, portrait, 11pt-12pt font, and enable page numbers and table of contents if offered.

——————————— COPY BELOW THIS LINE ———————————

Engineering Fundamentals: SOLID, DRY, KISS, API, Git/GitHub, Modular Architecture, REST

Author: Your Name
Version: 1.0
Paper: A4, Portrait

Overview

- SOLID: Five OO design principles for evolvable code.
- DRY: Single source of truth; remove duplication.
- KISS: Prefer the simplest design that fully solves the problem.
- API: Stable contracts for software interaction.
- Git/GitHub: Version control and collaboration workflow.
- Modular Architecture: Cohesive modules with explicit interfaces.
- REST: Resource-oriented HTTP design with standard verbs and status codes.

SOLID

Short note:

- SRP: One reason to change per module.
- OCP: Open for extension, closed for modification.
- LSP: Subtypes must honor base contracts.
- ISP: Many small interfaces over one fat interface.
- DIP: Depend on abstractions; invert concrete dependencies.

Real-life analogy:

- A restaurant kitchen: each station has a single job (SRP); new specials added without rewriting the menu (OCP); any trained chef can cover a station (LSP); focused toolsets per station (ISP); head chef defines recipes, line cooks follow steps (DIP).

Checklist:

- Keep classes narrowly focused.
- Extend via composition/plug-ins rather than editing stable code.
- Validate substitutability in tests.
- Split interfaces for specific clients.
- Use dependency injection for swappable implementations.

Code sketch:

```
// Ports and adapters (DIP/OCP/ISP)
export interface PaymentGateway { charge(cents: number): Promise<boolean>; }

export class StripeGateway implements PaymentGateway {
  async charge(cents: number) { /* call Stripe */ return true; }
}

export class OrderService {
  constructor(private gateway: PaymentGateway) {}
  async place(amount: number) {
    if (amount <= 0) throw new Error("invalid amount"); // SRP: business rule here
    return this.gateway.charge(amount);
  }
}
```

DRY

Short note:

- “Don’t Repeat Yourself”: Each rule or schema lives in one canonical place.

Real-life analogy:

- One master recipe card; update it once and everyone follows the same steps.

Checklist:

- Extract shared logic into functions/modules.
- Centralize constants, schemas, and validators.
- Avoid “false DRY” when things are similar but not the same.

Code sketch:

```
// Before: duplicated discount logic
function priceA(p){ return p * 0.9; }
function priceB(p){ return p * 0.8; }

// After: one parameterized rule
export function priceAfterDiscount(p, rate){ return p * (1 - rate); }
```

KISS

Short note:

- “Keep It Simple”: smallest solution that meets requirements; avoid over-engineering.

Real-life analogy:

- For a weekend trip, pack a backpack, not three suitcases.

Checklist:

- Small, readable functions.
- Clear naming over clever tricks.
- Add abstractions only with proven need.
- Optimize only after profiling.

Code sketch:

```
// KISS counter
type Counter struct { n int }
func (c *Counter) Inc(){ c.n++ }
```

API (general)

Short note:

- A contract: endpoints/methods, request/response shapes, errors, and versioning.

Real-life analogy:

- A restaurant menu: you order by contract; kitchen internals are hidden.

Checklist:

- Versioned endpoints (/v1).
- Consistent JSON shapes and error envelopes.
- Backward compatibility and deprecation policy.
- Authentication and rate limits defined.

Error envelope:

```
{
  "error": { "code": "VALIDATION_ERROR", "message": "Email invalid", "field": "email" }
}
```

Git and GitHub

Short note:

- Git manages history with commits and branches; GitHub adds hosting, PRs, and issues.

Real-life analogy:

- Git is your time machine and branching lab notebook; GitHub is the shared office.

Branching approaches:

- GitHub Flow: short-lived feature branches → PR → main.
- Trunk-based: tiny branches, frequent merges.

- GitFlow: develop, release, hotfix (more ceremony).

Core commands:

```
# New branch and push
git switch -c feature/auth
git push -u origin feature/auth

# Commit small and often
git add -A && git commit -m "feat(auth): register/login UI"

# Sync with main
git fetch origin
git rebase origin/main   # keep linear feature branch
# or: git merge origin/main

# Open PR on platform; squash merge if policy allows
```

PR hygiene:

- Small diff, clear title/description, linked issue, passing CI.
- Request review; address feedback; merge cleanly.

Issues:

- Use labels, assignees, milestones.
- Include steps to reproduce, expected vs actual, environment.
- Link issues to PRs for traceability.

Modular Architecture

Short note:

- System composed of cohesive modules with clear interfaces and minimal coupling.

Real-life analogy:

- PC with standardized connectors; swap the GPU without rebuilding the whole machine.

Design rules:

- Encapsulation: hide internals, expose a narrow API.
- Directional dependencies: avoid cycles.
- Stable interfaces, replaceable implementations.
- Separate domain, application, and infrastructure concerns.

Layer sketch:

```
[UI/Controllers] → [Application Services] → [Domain] → [Ports] ← [Adapters: DB, HTTP,
```

REST API Architecture

Short note:

- Resource-oriented HTTP using verbs, status codes, and stateless requests with consistent representations (JSON).

Resources and routes:

- Nouns: /users, /users/{id}, /orders/{id}/items
- Verbs:
  - GET: read
  - POST: create
  - PUT: replace
  - PATCH: partial update
  - DELETE: remove

Examples:

```
POST /v1/users
201 Created
Location: /v1/users/123
{
  "id": 123,
  "email": "a@b.com"
}
```

Status codes:

- 200 OK, 201 Created (+ Location), 204 No Content
- 400 Bad Request, 401 Unauthorized, 403 Forbidden
- 404 Not Found, 409 Conflict, 422 Unprocessable Entity
- 429 Too Many Requests, 500 Server Error

Design tips:

- Consistent pagination (page, pageSize or cursor).
- Filtering/sorting via query parameters.
- Idempotency for PUT/DELETE; idempotency keys for POST when needed.
- Versioning strategy: URI (/v1) or header-based.
- Auth: OAuth2/OIDC or PATs; prefer HTTP-only cookies for web sessions.

Putting It Together

- Start KISS to deliver a minimal working slice.
- Refactor to DRY as patterns emerge in validation, schemas, and queries.
- Introduce SOLID to define clean interfaces and extension points as complexity grows.
- Organize with modular architecture for team scaling and replaceable parts.
- Expose capabilities via a well-versioned REST API with strong contracts.
- Manage change with Git branches, PR reviews, and issues for traceability.

One-Page Cheatsheet (print)

- SOLID: SRP, OCP, LSP, ISP, DIP.
- DRY: One source of truth.
- KISS: Simplest working solution.
- REST verbs: GET, POST, PUT, PATCH, DELETE.
- Status codes: 200, 201(+Location), 204, 400, 401, 403, 404, 409, 422, 429, 500.
- Git flow: branch → commit small → PR → review → merge.
- Error shape: code, message, details; consistent JSON.
- Versioning: /v1; deprecate gracefully.

———————— STOP COPYING HERE ————————

Would you like this exported as:

- A4, 11pt, with a generated table of contents and page numbers, or
- Letter, 12pt, compact margins?

Reply with your preference, and the PDF will be produced and shared.

⁂

1. https://scalastic.io/en/solid-dry-kiss/
2. https://www.mindinventory.com/blog/software-design-principles/
3. https://idatamax.com/blog/engineering-with-solid-dry-kiss-yagni-and-grasp
4. https://www.docuwriter.ai/posts/mastering-software-design-principles-guide-building-better-software
5. https://www.geeksforgeeks.org/software-engineering/kiss-principle-in-software-development/
6. https://codefinity.com/blog/The-KISS-Principle
7. https://www.interaction-design.org/literature/topics/keep-it-simple-stupid
8. https://www.linkedin.com/posts/devanshu-verma07_softwaredesign-cleancode-solidprinciples-activity-7387511745793253376-CdUr
9. https://dev.to/kwereutosu/the-k-i-s-s-principle-in-programming-1jfg
10. https://www.reddit.com/r/programming/comments/1bmicj0/3_software_development_principles_i_wish_i_knew/

11. https://thevaluable.dev/kiss-principle-explained/
12. https://www.linkedin.com/posts/sina-riyahi_best-principles-in-software-design-software-activity-7383143698181644288-0wxx
13. https://www.pullchecklist.com/posts/clean-coding-principles
14. https://www.reddit.com/r/learnprogramming/comments/umr931/solid_dry_and_kiss/
15. https://www.reddit.com/r/coding/comments/jxsobw/solid_dry_kiss_principles_of_software_engineering/
16. https://www.baeldung.com/cs/kiss-software-design-principle
17. https://dev.to/yatendra2001/practical-insights-on-solid-dry-kiss-explained-in-noob-vs-pro-analogy-2a92
18. https://www.reddit.com/r/dotnet/comments/1384rhy/kiss_principle/
19. https://sysctl.id/keep-it-simple-stupid-principle-in-software-development/
20. https://en.wikipedia.org/wiki/KISS_principle
21. https://www.digitalocean.com/community/conceptual-articles/s-o-l-i-d-the-first-five-principles-of-object-oriented-design
22. https://blog.algomaster.io/p/solid-principles-explained-with-code
23. https://scalastic.io/en/solid-dry-kiss/
24. https://www.freecodecamp.org/news/introduction-to-solid-principles/
25. https://www.baeldung.com/cs/dry-software-design-principle
26. https://codefinity.com/blog/The-KISS-Principle
27. https://github.com/priyamshukla-coder/awesome-low-low-design/blob/main/concepts/KISS.md
28. https://github.com/harrymt/SOLID-principles
29. https://dev.to/accreditly/dry-what-is-it-and-how-to-implement-it-dont-repeat-yourself-59fj
30. https://blog.pixelfreestudio.com/best-practices-for-writing-dry-dont-repeat-yourself-code/
31. https://www.geeksforgeeks.org/system-design/solid-principle-in-programming-understand-with-real-life-examples/
32. https://dev.to/aneeqakhan/understanding-solid-design-principles-with-easy-coding-examples-1k1d
33. https://www.youtube.com/watch?v=_jDNAf3CzeY
34. https://www.reddit.com/r/dotnet/comments/1384rhy/kiss_principle/
35. https://www.reddit.com/r/ProgrammerTIL/comments/11tihj8/solid_design_principles_with_examples/
36. https://thevaluable.dev/dry-principle-cost-benefit-example/
37. https://www.freecodecamp.org/news/keep-it-simple-stupid-how-to-use-the-kiss-principle-in-design/
38. https://github.com/vikasg11/solid-principles
39. https://algomaster.io/learn/lld/kiss
40. https://dev.to/rachelsarchive/solid-principles-explained-with-examples-in-java-3d2g
41. https://www.skillvertex.com/blog/mern-stack-with-mysql/
42. https://www.bezkoder.com/node-js-express-sequelize-mysql/
43. https://blog.logrocket.com/build-rest-api-node-express-mysql/
44. https://www.mongodb.com/resources/languages/mern-stack
45. https://scalastic.io/en/solid-dry-kiss/

46. https://blog.stackademic.com/building-solid-express-js-applications-6d77e6a4d7d0
47. https://www.digitalocean.com/community/conceptual-articles/s-o-l-i-d-the-first-five-principles-of-object-oriented-design
48. https://hevodata.com/learn/nodejs-mysql-integration/
49. https://www.youtube.com/watch?v=-n1kCj5qRy0
50. https://github.com/jfullstackdev/mern-fullstack
51. https://github.com/bezkoder/node-js-express-login-mongodb
52. https://dev.to/yatendra2001/practical-insights-on-solid-dry-kiss-explained-in-noob-vs-pro-analogy-2a92
53. https://dev.to/corbado/build-a-full-stack-react-crud-app-with-nodejs-express-mysql-step-by-step-guide-hjk
54. https://strapi.io/blog/mern-stack-guide-components-setup-best-practices
55. https://ilarionhalushka.github.io/List-Of-Skills-Of-Node-JS-Developer/
56. https://algodaily.com/lessons/mern-stack-introduction-43fee134
57. https://dev.to/nknghiem/solid-kiss-yagni-and-dry-principles-ie7
58. https://github.com/SamehEtman/SOLID-Principles-JS
59. https://www.reddit.com/r/coding/comments/jxsobw/solid_dry_kiss_principles_of_software_engineering/
60. https://stackoverflow.com/questions/50919257/best-practices-with-mongodb-and-expressjs
61. https://pankajsharma.hashnode.dev/how-to-use-mysql-with-mern
62. https://tailwindcss.com/plus/ui-blocks/application-ui/forms/sign-in-forms
63. https://codingmstr.com/building-a-login-and-registration-form-with-validation-in-react-and-tailwind-css/
64. https://infinum.com/handbook/frontend/react/tailwind/best-practices
65. https://flowbite.com/docs/components/forms/
66. https://dev.to/deyemiobaa/adding-custom-validation-to-a-form-with-tailwindcss-1e7d
67. https://flowbite.com/blocks/marketing/register/
68. https://themeselection.com/tailwind-css-login-page-templates/
69. https://galaxies.dev/quickwin/react-tailwind-form
70. https://web3forms.com/blog/tailwindcss-form-validation
71. https://github.com/manuelson/react-login-template
72. https://tw-elements.com/docs/standard/forms/validation/
73. https://tw-elements.com/docs/react/forms/login-form/
74. https://www.loginradius.com/blog/engineering/guest-post/modern-login-signup-form-tailwindcss-react
75. https://tw-elements.com/docs/standard/forms/registration-form/
76. https://www.youtube.com/watch?v=8QgQKRcAUvM
77. https://dev.to/shubhamtiwari909/form-validation-with-js-and-tailwind-38ab
78. https://purecode.ai/community/authform-tailwind-authform
79. https://www.codewithfaraz.com/article/192/15-tailwind-signup-registration-form-examples
80. https://www.linkedin.com/posts/loginradius_build-a-modern-loginsignup-form-with-tailwind-activity-7262132086679969798-xsO2

81. https://www.freecodecamp.org/news/how-to-build-a-login-page-with-material-tailwind-framework/
82. https://www.atlassian.com/git/tutorials/merging-vs-rebasing
83. https://builtin.com/software-engineering-perspectives/git-rebase-vs-merge
84. https://www.simplilearn.com/git-rebase-vs-merge-article
85. https://graphite.dev/guides/git-checkout-vs-switch
86. https://stackoverflow.com/questions/57265785/whats-the-difference-between-git-switch-and-git-checkout-branch
87. https://refine.dev/blog/git-switch-and-git-checkout/
88. https://git-scm.com/docs/git-checkout
89. https://github.blog/developer-skills/github/how-to-create-issues-and-pull-requests-in-record-time-on-github/
90. https://docs.github.com/en/issues/tracking-your-work-with-issues/using-issues/viewing-all-of-your-issues-and-pull-requests
91. https://www.youtube.com/watch?v=0chZFIZLR_0
92. https://www.youtube.com/watch?v=g0NBVOiPFWA
93. https://www.youtube.com/watch?v=d3N2yeAaLkc
94. https://stackoverflow.com/questions/16666089/whats-the-difference-between-git-merge-and-git-rebase
95. https://www.reddit.com/r/git/comments/wkajik/rebase_vs_merge/
96. https://www.reddit.com/r/git/comments/1dzafey/merge_vs_rebase/
97. https://blog.git-init.com/differences-between-git-merge-and-rebase-and-why-you-should-care/
98. https://www.geeksforgeeks.org/git/git-difference-between-merging-and-rebasing/
99. https://dev.to/ccoveille/please-stop-using-git-checkout-learn-to-use-the-right-git-commands-an0
100. https://www.reddit.com/r/learnprogramming/comments/15fg3po/what_do_we_do_with_pull_requests_issues_in_github/
101. https://www.aviator.co/blog/rebase-vs-merge-pros-and-cons/
102. https://scalastic.io/en/solid-dry-kiss/
103. https://dev.to/nknghiem/solid-kiss-yagni-and-dry-principles-ie7
104. https://dev.to/yatendra2001/practical-insights-on-solid-dry-kiss-explained-in-noob-vs-pro-analogy-2a92
105. https://www.reddit.com/r/coding/comments/jxsobw/solid_dry_kiss_principles_of_software_engineering/
106. https://bnowdev.com/blog/understanding-solid-dry-and-kiss-in-software-development-with-typescript-examples/
107. https://alishoff.com/blog/487
108. https://christopherklint.com/blog/favorite-software-development-principles-solid-dry-kiss
109. https://www.linkedin.com/pulse/software-engineering-principles-applying-solid-dry-development
110. https://www.designgurus.io/blog/essential-software-design-principles-you-should-know-before-the-interview
111. https://www.reddit.com/r/programming/comments/1bmicj0/3_software_development_principles_i_wish_i_knew/

112. https://www.designgurus.io/blog/essential-software-design-principles-you-should-know-before-the-interview
113. https://bnowdev.com/blog/understanding-solid-dry-and-kiss-in-software-development-with-typescript-examples/
114. https://scalastic.io/en/solid-dry-kiss/
115. https://dev.to/nknghiem/solid-kiss-yagni-and-dry-principles-ie7
116. https://dev.to/yatendra2001/practical-insights-on-solid-dry-kiss-explained-in-noob-vs-pro-analogy-2a92
117. https://aws.amazon.com/what-is/restful-api/
118. https://mannhowie.com/rest-api
119. https://learn.microsoft.com/en-us/azure/architecture/best-practices/api-design
120. https://www.atlassian.com/git/tutorials/comparing-workflows/gitflow-workflow
121. https://docs.github.com/en/get-started/using-github/github-flow
122. https://www.datacamp.com/tutorial/git-branching-strategy-guide
123. https://learn.microsoft.com/en-us/azure/devops/repos/git/git-branching-guidance?view=azure-devops
124. https://restfulapi.net
125. https://dev.to/atanda0x/rest-api-best-practice-understanding-verbs-and-status-code-fei
126. https://dev.to/cassiocappellari/fundamentals-of-rest-api-2nag
127. https://pwskills.com/blog/git-workflow-devops/
128. https://blog.dreamfactory.com/rest-apis-an-overview-of-basic-principles
129. https://graphite.dev/guides/git-flow
130. https://datasciencedojo.com/blog/understanding-rest-api/
131. https://peerlist.io/teamcamp/articles/mastering-git-workflows-efficient-collaboration-guide-2025
132. https://www.reddit.com/r/learnprogramming/comments/17kkjas/what_actually_is_a_rest_api_can_someone_provide/
133. https://pradeepl.com/blog/rest/rest-communicating-with-verbs-and-status-codes/
134. https://dev.to/ajmal_hasan/beginner-friendly-git-workflow-for-developers-2g3g
135. https://blog.prateekjain.dev/the-ultimate-guide-to-git-branching-strategies-6324f1aceac2
136. https://www.scribd.com/document/887358823/Git-Branch-Pull-Request-Workflow-Guide
137. https://www.pdfforge.org/online/en/markdown-to-pdf
138. https://www.reddit.com/r/Markdown/comments/1hrssk7/markdown_to_pdf_converter/
139. https://apitemplate.io/pdf-tools/convert-markdown-to-pdf/
140. https://md-to-pdf.fly.dev
141. https://cloudconvert.com/md-to-pdf
142. https://www.pdfforge.org/online/en/markdown-to-pdf?srsltid=AfmBOopi8ScE8e3QCHbKYk-TJHfiAR2WowcDKnybJ1CZHUSMxvgxBVXu
143. https://lightpdf.com/markdown-to-pdf
144. https://markdownlivepreview.dev/tools/markdown-to-pdf
145. https://linangdata.com/pdf-generator/
146. https://www.reddit.com/r/Markdown/comments/pm5n3m/how_to_convert_markdown_file_to_pdf/